



**Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)**

ФАКУЛЬТЕТ

«Информатика и системы управления»

КАФЕДРА

«Теоретическая информатика и компьютерные технологии»

**Отчет о лабораторной работе № 4
по курсу «Операционные системы»
Операции с виртуальной памятью**

Студент: Федукон А. А.

Группа: ИУ9-42Б

Преподаватель: Брагин А.В.

Москва, 2025 г.

Содержание

Постановка задачи	2
Часть 1. Работа с памятью в режиме ядра в операционной системе семейства Windows NT	2
Часть 2. Работа с памятью в режиме ядра в NetBSD	3
Практическая реализация	3
Часть 1. Работа с памятью в режиме ядра в операционной системе семейства Windows NT	3
Часть 2. Работа с памятью в режиме ядра в NetBSD	6
Результаты	8
Часть 1. Работа с памятью в режиме ядра в операционной системе семейства Windows NT	8
Часть 2. Работа с памятью в режиме ядра в NetBSD	9
Выводы	9
Список литературы	9

Постановка задачи

Цель работы — разработать загружаемые модули ядра (драйвера) для ОС Windows или ReactOS и NetBSD, которые выводят в отладочный журнал список всех процессов в системе.

Часть 1. Работа с памятью в режиме ядра в операционной системе семейства Windows NT

Используя созданный в лабораторной работе № 2 минимальный драйвер, совместимый с ОС Windows NT/ReactOS и NetBSD, реализовать следующие действия с виртуальной памятью.

- Зарезервировать 10 страниц виртуальной памяти с использованием функции `ZwAllocateVirtualMemory` с флагом `MEM_RESERVE`;
- Инициализировать (выделить физическую память) для первых 5 страниц с флагом `MEM_COMMIT`;
- Вывести физические адреса и значения Page Table Entries (PTE) для всех 10 страниц;
- Освободить выделенную память.

Часть 2. Работа с памятью в режиме ядра в NetBSD

Используя созданный в лабораторной работе №2 драйвер, реализовать следующие действия с виртуальной памятью.

1. Зарезервировать 10 страниц виртуальной памяти, используя функцию ядра `uvm_km_alloc`.
2. Обеспечить пять первых страниц из выделенных 10 физическими страницами памяти, используя функции ядра `uvm_pglstalloc` и `pmap_kenter_pa`.
3. Вывести физические адреса и значения PTE для всех 10 страниц в шестнадцатеричном формате.
4. Освободить выделенную память, используя функции ядра `pmap_kremove`, `pmap_update`, `uvm_pglstfree`, `uvm_km_free`.

Драйвер должен выгружаться динамически с помощью команды `modunload`.

Практическая реализация

Часть 1. Работа с памятью в режиме ядра в операционной системе семейства Windows NT

Для выполнения лабораторной работы была использована структура исходного кода драйвера из лабораторной работы №2. Папка `lab2` была скопирована

в drivers/lab4, после чего файлы lab2.c и lab2.rc были переименованы в lab4.c и lab4.rc соответственно. В файл drivers/CMakeLists.txt добавлена строка:

```
add_subdirectory(lab4)
```

Также были скорректированы пути и имена исходных файлов в drivers/lab4/CMakeLists.txt.

В коде драйвера были реализованы все действия, указанные в постановке задачи. Имя устройства и символьной ссылки были изменены на персонализированные, содержащие фамилию автора:

```
#define DEVICE_NAME L"\\Device\\FedukovLab4"
#define SYMBOLIC_LINK_NAME L"\\DosDevices\\FedukovLab4"
```

Создание устройства и символьной ссылки производится через функции IoCreateDevice и IoCreateSymbolicLink.

Далее была зарезервирована область виртуальной памяти размером 10 страниц с помощью вызова ZwAllocateVirtualMemory с флагом MEM_RESERVE [3]:

```
size = PAGE_SIZE * 10;
status = ZwAllocateVirtualMemory(
    NtCurrentProcess(),
    &addr,
    0,
    &size,
    MEM_RESERVE,
    PAGE_READWRITE);
```

После этого были закомментированы первые 5 страниц (выделена физическая память):

```
size = PAGE_SIZE * 5;
status = ZwAllocateVirtualMemory(
    NtCurrentProcess(),
    &addr,
```

```

0,
&size,
MEM_COMMIT,
PAGE_READWRITE);

```

Для получения информации о таблице страниц (PTE) были определены вспомогательные макросы [4]:

```

#define PTE_OFFSET 0xC0000000
#define PTE_addr(X) (PHARDWARE_PTE_X86)((ULONG)(PTE_OFFSET) + (X))
#define PTE_next(X) (PHARDWARE_PTE_X86)((ULONG)(X) + sizeof(PHARDWARE_PTE_X86))

```

Итерируясь по всем 10 страницам, драйвер извлекает поля Valid, Dirty и физический адрес каждой страницы (вычисляется через сдвиг номера страницы PageFrameNumber на 12 бит):

```

ppte = PTE_addr(addr);
for (it = 0; it < 10; it++) {
    if (it < 5) {
        access = *(volatile SIZE_T*)((ULONG_PTR)addr + PAGE_SIZE * it);
        UNREFERENCED_PARAMETER(access);
    }
    DPRINT1("Page: %d\n", (int)it);
    DPRINT1("Valid: %d\n", ppte->Valid);
    DPRINT1("Dirty: %d\n", ppte->Dirty);
    DPRINT1("Physical address: %X\n", (ppte->PageFrameNumber) << 12);
    ppte = PTE_next(ppte);
}

```

Завершив работу, драйвер освобождает память через ZwFreeVirtualMemory с флагом MEM_RELEASE:

```

size = 0;
ZwFreeVirtualMemory(NtCurrentProcess(), &addr, &size, MEM_RELEASE);

```

Также реализована функция DriverUnload, в которой удаляются символьная ссылка и устройство, созданные при инициализации.

Сборка образа системы ReactOS с драйвером производилась с помощью команды `ninja bootcd`. Далее образ был установлен в виртуальную машину с использованием QEMU.

Загрузка драйвера в системе ReactOS осуществлялась аналогично лабораторной работе №3: через встроенную утилиту `sc`:

```
sc create lab4driver binPath=  
C:\ReactOS\system32\drivers\lab4.sys type= kernel
```

```
sc start lab4driver
```

После загрузки драйвер выводил отладочную информацию через `DPRINT1`, которая просматривалась в отладчике системы или в логах виртуальной машины.

Часть 2. Работа с памятью в режиме ядра в NetBSD

Для выполнения второй части лабораторной работы №4 я использовал ранее созданный модуль ядра из лабораторной работы №2. В каталоге `/usr/src/sys/modules/` была создана новая директория `lab4`, в которую я скопировал содержимое директории `lab2`. После этого файл `lab2.c` был переименован в `lab4.c`, а в `Makefile` соответствующим образом обновлены имена исходных файлов и целевого модуля.

Основной задачей было выделить виртуальное адресное пространство и отобразить в него физическую память, используя механизмы ядра NetBSD. В начале функции инициализации модуля `lab4_modcmd` с помощью функции `uvm_km_alloc` я зарезервировал 10 страниц виртуальной памяти:

```
vaddr = uvm_km_alloc(kernel_map, NPAGES * PAGE_SIZE, 0,  
UVM_KMF_VAONLY | UVM_KMF_WAITVA);
```

Поскольку флаг `UVM_KMF_VAONLY` означает, что физическая память не выделяется, на этом этапе страницы существуют только виртуально. Сразу после этого я вывел информацию о каждой из 10 страниц, включая флаги `Valid`,

Used, Modified и физический адрес (если отображён), с помощью вспомогательной функции `print_page_info [?]`.

Затем с помощью функции `uvm_pglistalloc` я попытался выделить 5 физических страниц, которые затем нужно было отобразить в ранее зарезервированное виртуальное пространство:

```
uvm_pglistalloc(COMMIT_PAGES * PAGE_SIZE, 0, ~0L, 0, 0,
                &phys_pages, COMMIT_PAGES, 0);
```

Выделенные страницы из списка `phys_pages` были отображены в первые 5 страниц виртуального пространства через функцию `pmap_kenter_pa`:

```
pmap_kenter_pa(vaddr + (i * PAGE_SIZE),
               VM_PAGE_TO_PHYS(pg),
               VM_PROT_READ | VM_PROT_WRITE, 0);
```

После завершения отображения я вызвал `pmap_update(pmap_kernel())`, чтобы синхронизировать изменения с таблицей отображений ядра. Затем снова вывел состояние всех 10 страниц в консоль, чтобы отследить изменения в статусе и отображении памяти.

Особое внимание я уделил правильному перебору списка `phys_pages`. Поскольку `TAILQ_FOREACH` используется для обхода двусвязного списка, важно использовать корректное имя очереди (`pageq.queue`), чтобы получить доступ к структурам `vm_page`.

При выгрузке модуля (обработка `MODULE_CMD_FINI`) я последовательно:

- удалил отображения 5 физических страниц с помощью `pmap_kremove`;
- вызвал `pmap_update` для применения изменений;
- освободил физические страницы с помощью `uvm_pglistfree`;
- освободил виртуальное адресное пространство через `uvm_km_free`.

Завершив реализацию, я выполнил сборку модуля в директории `/usr/src/sys/modules/lab4` командой [3]:

make

Загрузку драйвера я производил вручную, используя следующую команду:

```
modload ./lab4.kmod
```

Вывод драйвера отображался в консоли и подтверждал правильность резервирования, отображения и освобождения памяти. Модуль успешно выгружался с помощью команды:

```
modunload lab4
```

Результаты

Таким образом, в рамках лабораторной работы были успешно реализованы все требуемые действия с виртуальной памятью в режиме ядра, получен доступ к таблице страниц и выведена информация о виртуально-физическом отображении памяти.

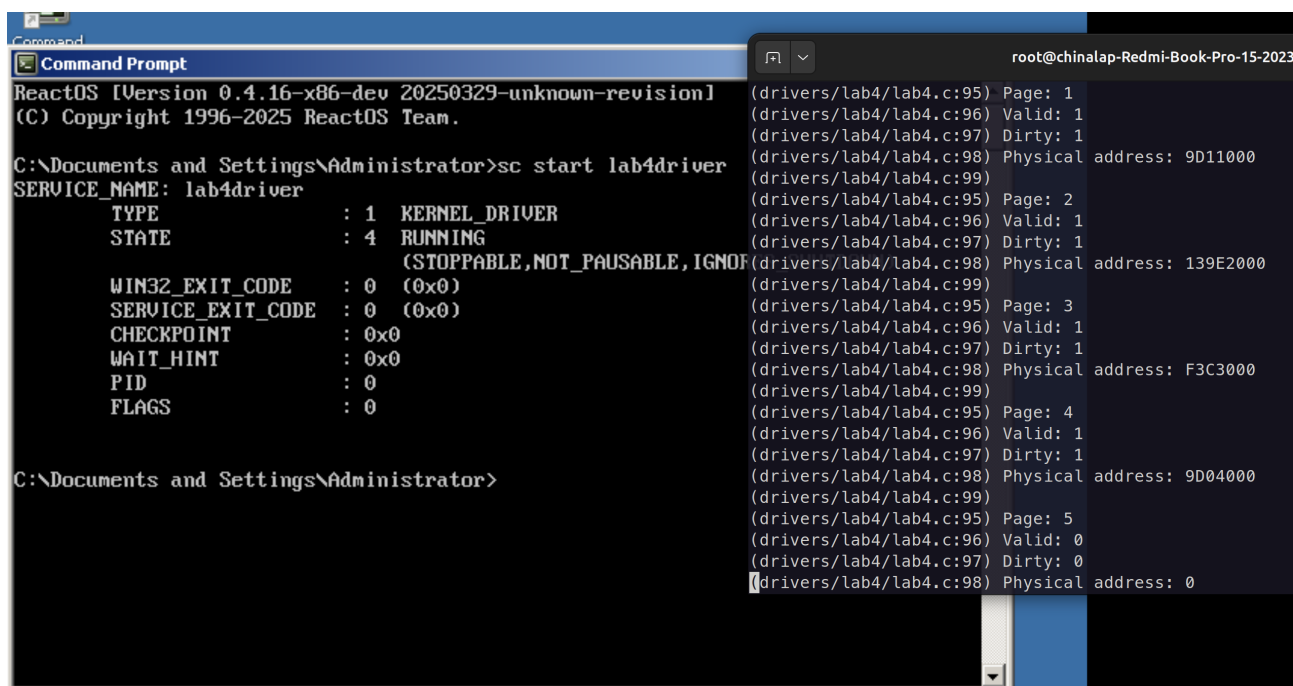
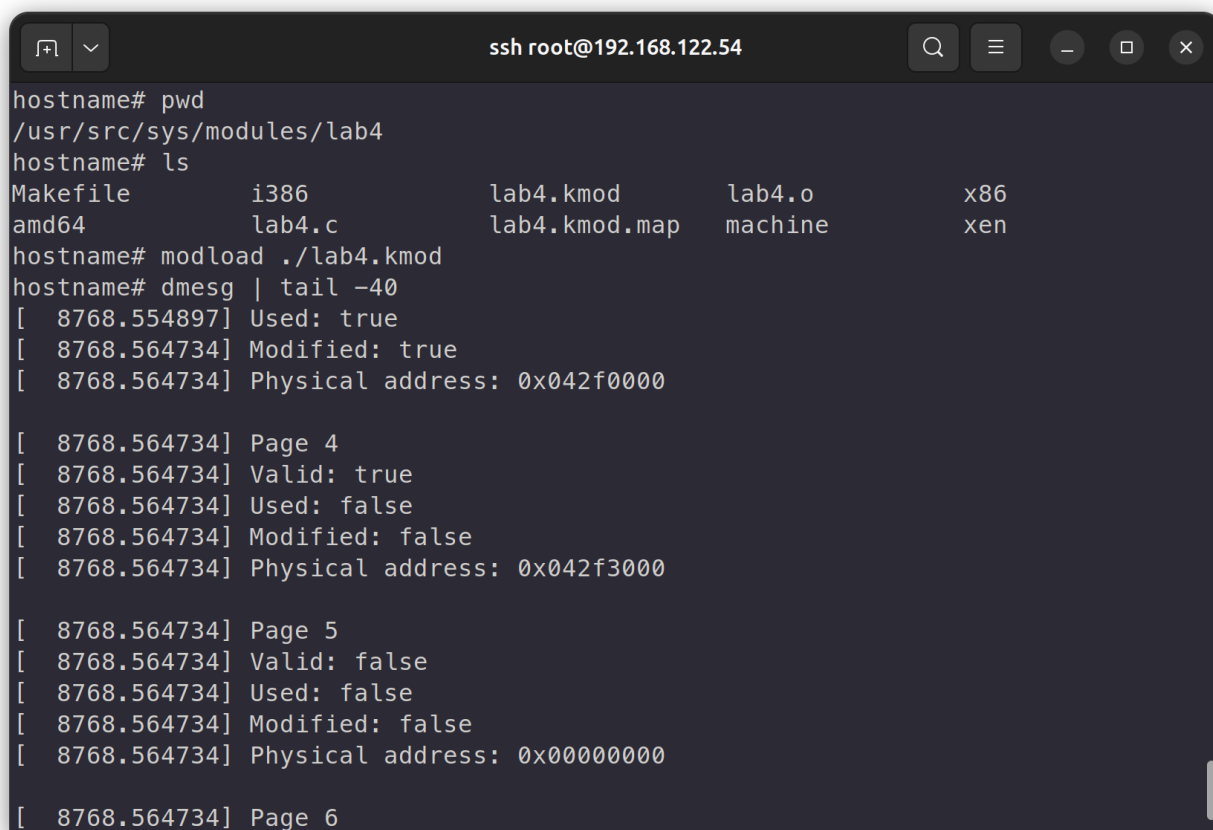


Рис. 1 — Процессы в reactos

A terminal window titled 'ssh root@192.168.122.54' with standard window controls. The terminal shows the following commands and output:

```
hostname# pwd
/usr/src/sys/modules/lab4
hostname# ls
Makefile          i386          lab4.kmod      lab4.o        x86
amd64             lab4.c        lab4.kmod.map  machine       xen
hostname# modload ./lab4.kmod
hostname# dmesg | tail -40
[ 8768.554897] Used: true
[ 8768.564734] Modified: true
[ 8768.564734] Physical address: 0x042f0000

[ 8768.564734] Page 4
[ 8768.564734] Valid: true
[ 8768.564734] Used: false
[ 8768.564734] Modified: false
[ 8768.564734] Physical address: 0x042f3000

[ 8768.564734] Page 5
[ 8768.564734] Valid: false
[ 8768.564734] Used: false
[ 8768.564734] Modified: false
[ 8768.564734] Physical address: 0x00000000

[ 8768.564734] Page 6
```

Рис. 2 — Процессы в netbsd

Выводы

В ходе выполнения второй части лабораторной работы я на практике познакомился с механизмами работы с виртуальной памятью в ядре операционной системы NetBSD. Мне удалось реализовать резервирование виртуального адресного пространства, выделение физической памяти, отображение страниц и последующее освобождение ресурсов.

Было полезно увидеть, как логическое (виртуальное) адресное пространство связывается с физической памятью средствами ядра, а также как это отображается через PTE (Page Table Entries).

Работа с разными макросами и функциями ядра NetBSD расширила моё понимание структуры и механики управления памятью, что может пригодиться для дальнейшего изучения операционных систем и разработки драйверов.

Список литературы

- [1] Официальная документация ReactOS
(<https://reactos.org/wiki/>)
- [2] Официальная документация NetBSD
(<https://www.netbsd.org/docs/>)
- [3] Учебно-методическое пособие курса "Операционные системы"
(<https://press.bmstu.ru/catalog/item/8226/>)
- [4] Рекомендованное пособие по реализации драйверов в ReactOS
(https://doc.lagout.org/operating%20system%20/Windows/Memory_Scanning_Unde)